

Regression

Overview

No Class Meeting (MLK Day)

- a **machine learning** task where we **predict** a real-valued **output** response given an **input vector**
- **linear** and **non-linear** regression **models**
- **analytic** (closed-form) **solution** to learn an **optimal model** from **training data**
- **gradient descent** and **stochastic gradient** descent algorithms
- model **complexity** and **overfitting**

Readings

- <https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf#page=157>

Bishop, Pattern Recognition and Machine Learning (2006) pp. 137-152

Key Concepts

Linear Model	a function that adds up weighted inputs
Loss	a measure of how wrong predictions are
Least Squares	find weights that make loss as small as possible
Basis Function	transforms input so models can capture more complex patterns
Maximum Likelihood	statistical way to find weights that make observed data most probable
Predictive Distribution	range of possible outputs with associated confidence

Scope of Reading

- **fit** a linear **model** to **data**
- **measure** model **efficacy**
- **interpret** the **model** statistically

Linear Regression Model	- the linear model predicts $y(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_Dx_D$ where: $x = [x_1, x_2, \dots, x_D]$ is a vector of inputs. $w = (w_0, w_1, \dots, w_D)$ are parameters (weights). w_0 is the bias (or intercept) - each w_j scales the corresponding input feature
Loss Function: Sum of Squared Errors	- to train the model, we choose weights that make predictions close to real targets measure each data point error = predicted value – actual value - sum squared errors over all data points $E(w) = \sum_{n=1}^N (y(x_n) - t_n)^2$ advantages - all errors are positive large errors are penalized more
Finding the Best Weights (Least Squares)	- set the derivative of error with respect to weights to zero - produces a formula that can be solved in a single step - in matrix form - let Φ = design matrix (features for all data points) - let $\mathbf{t}$ = vector of target values optimal weights: $w^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$
Geometric Interpretation	- the model attempts to find the closest fit of a hyperplane to the data points - the error vector is orthogonal to the column space of the design matrix - we project target values onto the space spanned by basis functions

Basis Functions	- nonlinear relationships can be modeled linearly in transformed space using basis functions polynomial: $\phi_j(x) = x^j$ Gaussian: $\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2s_j^2}\right)$ - the model becomes: $y(x) = \sum_j w_j \phi_j(x)$
Probabilistic Interpretation	- we assume targets are generated by model plus Gaussian noise with zero mean and variance σ^2 - likely: $p(t_n x_n, w, \beta) = \mathcal{N}(t_n w^T \phi(x_n), \beta^{-1})$ - where: $\beta = 1/\sigma^2$ is the noise precision (this turns least squares into maximum likelihood estimation)
Maximum Likelihood for Weights	- under Gaussian noise, the likelihood of the entire dataset is the product of individual Gaussian terms maximizing likelihood is equivalent to minimizing squared errors least-squares solution = maximum likelihood solution
Predictive Distribution	- once we estimate weights, predicted distribution of a new input x_* is Gaussian - mean: $y(x_*) = w_T \phi(x_*)$ variance depends on noise variance and how far x_* is from training inputs - this gives prediction, and confidence in prediction
Overfitting and Regularization	- if the model has too many parameters it may fit noise instead of the true pattern - to reduce overfitting - add a penalty for large weights (regularization) - this can be interpreted as having a prior on weights

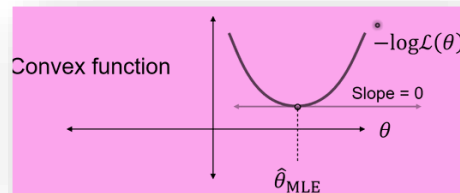
	• regularization solution balances the fit to data and simplicity of model
Async	
Linear Regression	
Definition	• supervised learning technique used to train models with multiple input variables mapped to a single Real value • we map the vectorized inputs to a corresponding output
1-D and Multi-D	• one-dimensional: models a real valued input to a real variable response so that, $y = mx + b$ • multi-dimensional: models vectorized real values to a single response variable, $\hat{y} = \beta_{i1}x_1 + \beta_{i2}x_2 + \beta_0$
MLE Roadmap	MLE = 'maximum likelihood estimation' • <i>the roadmap</i> 1. establish model likelihood for data 2. derive an analytical formula for optimal parameters on training data 3. extend to a nonlinear model • $\arg \max_{\theta} \prod_{i=1}^n p\theta(y_i x_i)$ supervised task, IID data (IID: Independent and Identically Distributed Data; for n data points: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$) • if we set up the maximum likelihood estimation task for linear regression will be equivalent for linear regression to minimizing the sum of squared errors • $\arg \max_{\theta} \prod_{i=1}^n p\theta(y_i x_i)$ $= \arg \max_{\theta \in \mathbb{R}^p} - \sum_{i=1}^n (y_i - \theta^T x_i)^2$

	$= \underset{\theta \in \mathbb{R}^p}{\arg \min} \sum_{i=1}^n (y_i - \theta^T x_i)^2$
Model and Likelihood	• Linear Probabilistic Model: $y = \theta^T x + \varepsilon$, where ○ θ^T is the vector of parameters ○ x is the vector of covariances ○ and ε is real value noise of the model: $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ • linear combination of covariates: $\sum_{i=1}^p \theta_i x_i$
Conditional Likelihood	• Conditioned on x, the response variable y follows a Gaussian distribution with mean $\theta^T x$ and variance σ^2 • because we are adding a mean zero noise, and we know that adding a constant to a Gaussian is equivalent to changing its mean, the Gaussian goes from zero to the added constant • so, y, given x, will be a Gaussian centered at the linear prediction with variance given by sigma squared, $Y \sim \mathcal{N}(\theta^T x, \sigma^2)$
MLE with Dataset	• Estimating the Model ○ given data, how can we estimate θ? ○ construct maximum likelihood estimator (MLE) ▪ derive the log-likelihood ▪ find θ MLE that maximizes likelihood • analytically: take derivative and set to 0 • iteratively: (stochastic) gradient descent • defining the likelihood ○ conditional density: $p_{\theta}(y \mid x) = \left(\frac{1}{\sigma\sqrt{2\pi}} \right) \cdot \exp\left(-\frac{(y - \theta^T x)^2}{2\sigma^2}\right)$ ○ likelihood function: $\mathcal{L}(\theta; \mathcal{D}) = \prod_{i=1}^n p_{\theta}(y_i \mid x_i) = \prod_{i=1}^n \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(-\frac{(y_i - \theta^T x_i)^2}{2\sigma^2}\right) =$

$$\frac{1}{\sigma^n (2\pi)^{\frac{n}{2}}} \exp\left(\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \theta^T x_i)^2\right)$$

Analytical Solution

- minimizing the **likelihood** (minimizing the squared error) is a **convex function** where we **seek** the slope (**gradient**) = 0



- the **gradient** is generalizing the **derivative** for **multivariate functions**

$$-\nabla_{\theta} \log \mathcal{L}(\theta) = \nabla_{\theta} \sum_{i=1}^n (y_i - \theta^T x_i)^2$$

Chain rule →

$$= -2 \sum_{i=1}^n (y_i - \theta^T x_i) x_i$$

$$= -2 \sum_{i=1}^n y_i x_i + 2 \sum_{i=1}^n (\theta^T x_i) x_i$$

- rewriting with **matrix** notation

- represent data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ as,

covariate (design) **matrix**:

$$\begin{matrix} n \\ \left\{ \right. \end{matrix} X = \underbrace{\begin{pmatrix} -x_1- \\ -x_2- \\ \vdots \\ -x_n- \end{pmatrix}}_p \in \mathbb{R}^{np}$$

assume X has rank p (not degenerate)

response vector

$$\begin{matrix} n \\ \left\{ \right. \end{matrix} Y = \underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}}_1 \in \mathbb{R}^n$$

- rewriting the **gradient** in matrix form

	$-\nabla \theta \log \mathcal{L}(\theta) = -2 \sum_{i=1}^n y_i x_i + 2 \sum_{i=1}^n (\theta^T x_i) x_i$ $= -2 X^T Y + 2 X^T X \theta$ our goal is gradient = 0 which gives,$X_T X \theta = X_T Y$$\theta = (X_T X)^{-1} (X_T Y)$																												
Computing with MLE Parameters	- computing the MLE: $\hat{\theta}_{MLE} = (X^T X)^{-1} X^T Y$ - not typically solved by inverting $X_T X$ - solved using direct methods: Cholesky factorization - up to a factor of 2 faster QR factorization more numerically stable - solved using various iterative methods: (stochastic) gradient descent or use the built-in solver in your math library R: <code>solve(Xt %*% X, Xt %*% y)</code>																												
Cholesky Factorization	solve for: $\hat{\theta}_{MLE} (X^T X) \hat{\theta}_{MLE} = X^T Y$ - compute symm. matrix $C = X^T X$ $O(np^2)$ - compute vector $d = X^T Y$ $O(np)$ Cholesky factorization L is lower triangular $LL^T = C$ $O(p^3)$ forward subs. to solve: $Lz = d$ $O(P^2)$ backward subs. to solve: $L^T \hat{\theta}_{MLE} = z$ $O(P^2)$																												
Solving Triangular System	<table><tr><td>A_{11}</td><td>A_{12}</td><td>A_{13}</td><td>A_{14}</td><td></td><td>x_1</td><td>b_1</td></tr><tr><td></td><td>A_{22}</td><td>A_{23}</td><td>A_{24}</td><td>*</td><td>x_2</td><td>b_2</td></tr><tr><td></td><td></td><td>A_{33}</td><td>A_{34}</td><td></td><td>x_3</td><td>b_3</td></tr><tr><td></td><td></td><td></td><td>A_{44}</td><td></td><td>x_4</td><td>b_4</td></tr></table>	A_{11}	A_{12}	A_{13}	A_{14}		x_1	b_1		A_{22}	A_{23}	A_{24}	*	x_2	b_2			A_{33}	A_{34}		x_3	b_3				A_{44}		x_4	b_4
A_{11}	A_{12}	A_{13}	A_{14}		x_1	b_1																							
	A_{22}	A_{23}	A_{24}	*	x_2	b_2																							
		A_{33}	A_{34}		x_3	b_3																							
			A_{44}		x_4	b_4																							

$A_{11}x_1$	$A_{12}x_2$	$A_{13}x_3$	$A_{14}x_4$
	A_{22}	A_{23}	A_{24}
		A_{33}	A_{34}
			A_{44}

$$x_1 = \frac{b_1 - A_{12}x_2 - A_{13}x_3 - A_{14}x_4}{A_{11}}$$

$$x_2 = \frac{b_2 - A_{23}x_3 - A_{24}x_4}{A_{22}}$$

$$x_3 = \frac{b_3 - A_{34}x_4}{A_{33}}$$

$$x_4 = \frac{b_4}{A_{44}}$$

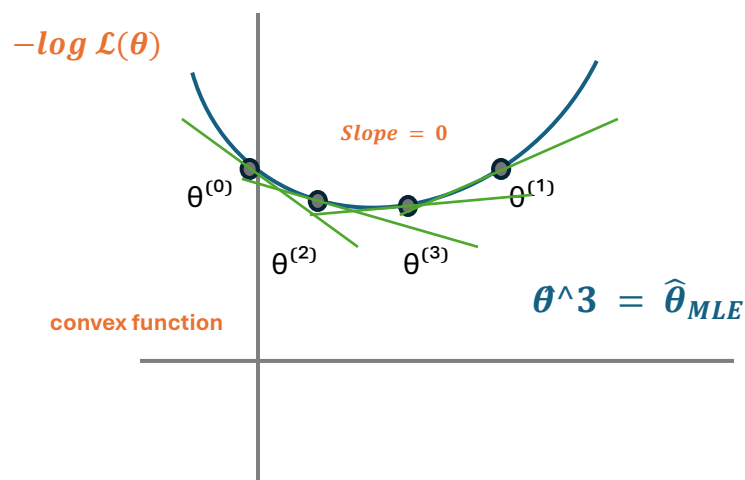
Iterative Optimization

- **Gradient descent** is an **iterative operation** concerned with **minimizing** some **loss** with respect to certain **parameters**
- Minimize $L(\theta)$ with respect to θ

For τ from 0 to convergence:

$$\theta^{\tau+1} = \theta^{\tau} - \underbrace{\rho(\tau)}_{\text{learning rate}} \underbrace{\nabla L(\theta^{\tau})}_{\text{direction of steepest ascent}}$$

Gradient Descent Illustrated



**Speeding Up
Gradient
Descent**

- What if P is large?

- the cost of $O(np^2) = O(n^3)$ could be prohibitive

for τ from 0 to convergence

$$\theta^{(t+1)} = \theta^{(t)} - \rho(\tau) \nabla_{\theta} \log \mathcal{L}(\theta^{(t)} | \mathcal{D})$$

$$\propto \theta^{(t)} - \rho(\tau) \nabla_{\theta} \sum_{i=1}^n (y_i - \theta^T x_i)^2$$

$$= \theta^{(t)} + \rho(\tau) \cdot 2 \sum_{i=1}^n (y_i - \theta^T x_i) x_i \quad O(np)$$

- can we do better?

- Solution: iterative methods

- gradient descent:

$$L(\theta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \theta^T x_i)^2$$

for τ from 0 to convergence

$$\theta^{t+1} = \theta^t - \rho(\tau) \nabla L(\theta^t)$$

$$= \theta^t + \rho(\tau) \cdot \underbrace{\frac{1}{n} \sum_{i=1}^n (y_i - \theta^T x_i) x_i}_{\text{estimate of the gradient}}$$

**Stochastic
Gradient
Descent**

- decomposable loss

- loss can be written as a sum of the loss on each record

$$L(\theta) = \sum_{i=1}^n L_i(\theta) = \sum_{i=1}^n L(\theta, x_i, y_i)$$

- equivalent to minimize

$$\frac{1}{n} L(\theta) = \frac{1}{n} \sum_{i=1}^n L(\theta, x_i, y_i)$$

- with gradients

$$\frac{1}{n} \nabla_{\theta} L(\theta) = \frac{1}{n} \sum_{i \in B} \nabla_{\theta} L(\theta, x_i, y_i) \approx \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} L(\theta, x_i, y_i)$$

$B \sim \text{random subset of indices}$

- **decomposable loss:**

$$L(\theta) = \sum_{i=1}^n L_i(\theta) = \sum_{i=1}^n L(\theta, x_i, y_i)$$

loss can be written as a sum of the loss on each record

- **visualizing gradient descent**

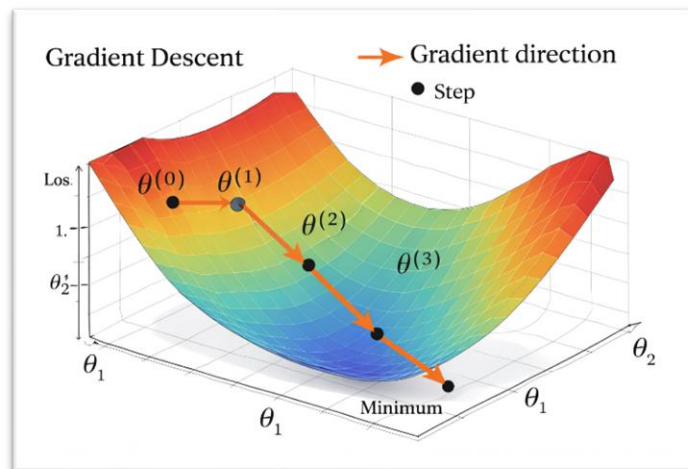


Image generated with ChatGPT (GPT-5.2), OpenAI. Joe Waller.

Stochastic Gradient Descent for Linear Regression

- for linear regression, **stochastic gradient descent** is just a matter of **randomly subsampling indices** and then **computing gradient**
- **construct a noisy estimate of the gradient:**
- for τ from **0** to **convergence**
 1. pick a **random i**
 2. $\theta^{\tau+1} = \theta^{\tau} + \rho(\tau) (y_i - \theta^{(\tau)T} x_i) x_i$ $O(np)$
- applies to **streaming data $O(p)$ storage**
- optimizing using a **single data point** at a time opens the door for **‘online’ or ‘streaming’ optimization**
- this **ability** to only consider **select data points** at a time comes in especially **useful** when **training a large model** with **limited hardware**

Nonlinear Regression

- **linear regression** can represent nonlinear functions by **applying** nonlinear **feature transformations** and **modeling** the output as a **linear combination** of those **features**
- **representing** the design **matrix** on our chosen **feature space**,
- train:

$$\Phi(X): n \times k \quad \Phi(X) = \begin{bmatrix} -\varphi(x^1) - \\ -\varphi(x^2) - \\ \vdots \\ -\varphi(x_n) - \end{bmatrix}$$

$$\hat{\theta} \leftarrow \text{solve}_{\theta}(\Phi(X)^T \Phi(X) \theta = \Phi(X)^T Y)$$

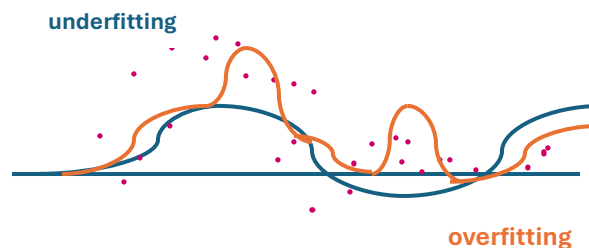
- test:

$$x': p \times 1 \quad \varphi(x'): k \times 1$$

$$\circ \hat{y} \leftarrow \theta^T \varphi(x')$$

Overfitting and Model Variance

- **overfitting** can give a **false sense** of doing “really well”
- **Underfitting** occurs when a machine learning **model** is too **simple** to capture the underlying **patterns** in **data**, resulting in **poor performance** on both **training** and **testing** datasets



**Regularization to
Prevent
Overfitting**

-

Live Session Notes		30 Oct 2025
Project Proposal	• due 02/11 • deliverables ○ state the task ○ state the goals	
*	• • •	
*	• • •	
*	• • •	